CSE 371: Database Systems

جامعة مصر للمعلوماتية
EGYPT UNIVERSITY
OF INFORMATICS

Spring 2024

FACULTY OF
ENGINEERING

Prof. Hala Zayed

جامعة مصر للمعلوماتية
EGYPT UNIVERSITY
OF INFORMATICS

06 – Normalization

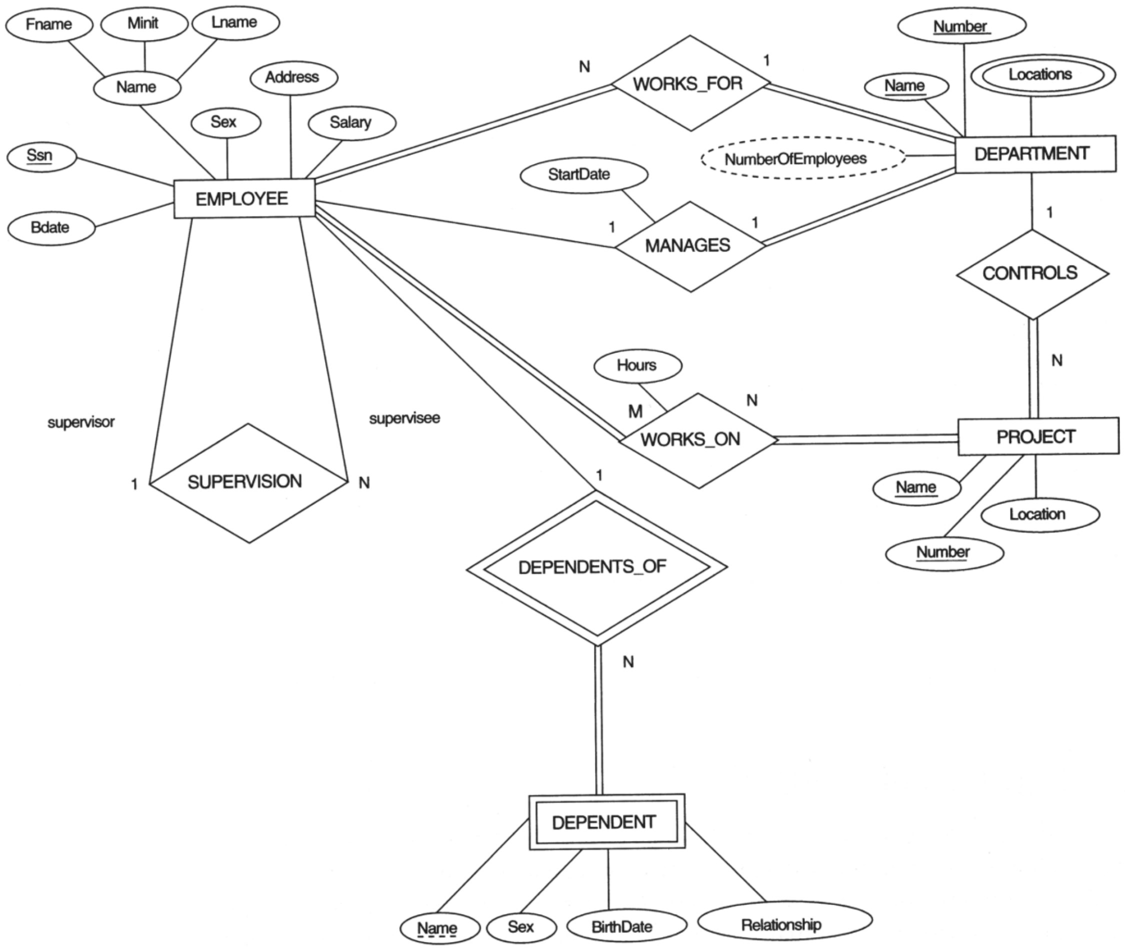
جامعة مصر للمعلوماتية
EGYPT UNIVERSITY
OF INFORMATICS

FACULTY OF
ENGINEERING

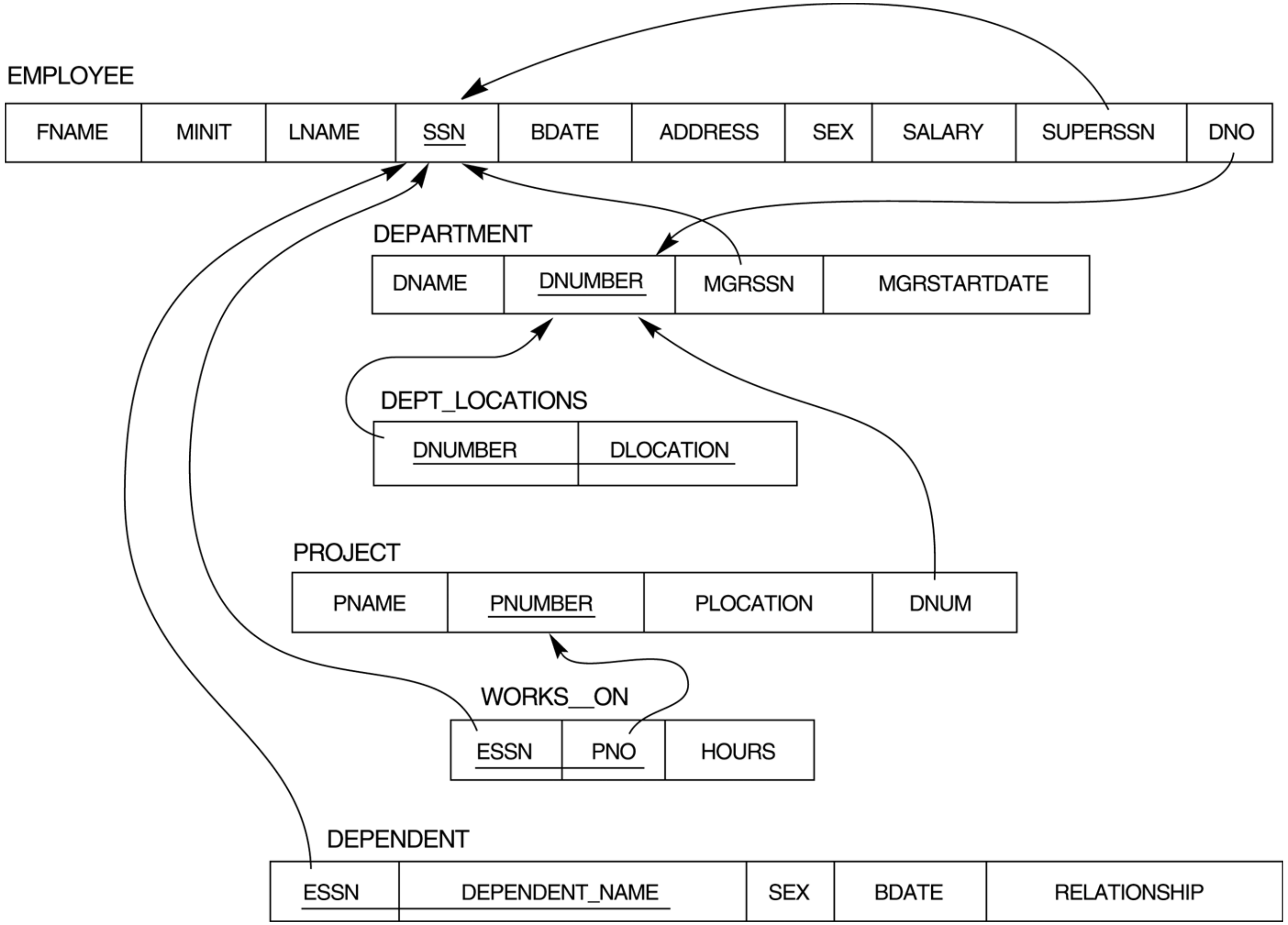
Contents

Contents

The ER conceptual schema diagram for the COMPANY database.



Result of mapping the COMPANY ER schema into a relational schema.



1. Informal Design Guidelines for Relational Databases

جامعة مصر للمعلوماتية
EGYPT UNIVERSITY
OF INFORMATICS

- Before discussing the formal theory of relational database design, we discuss four informal guidelines that may be used as measures to determine the quality of relation schema design:
  - Making sure that the semantics of the attributes is clear in the schema
  - Reducing the redundant information in tuples
  - Reducing the NULL values in tuples
  - Disallowing the possibility of generating spurious tuples
- These measures are not always independent of one another

1. Informal Design Guidelines for Relational Databases

- Then we discuss formal concepts of functional dependencies and normal forms
  - » - 1NF (First Normal Form)
  - » - 2NF (Second Normal Form)
  - » - 3NF (Third Normal Form)
  - » - BCNF (Boyce-Codd Normal Form)

1.1 Semantics of the Relational Attributes must be clear

- GUIDELINE 1: Informally, each tuple in a relation should represent one entity or relationship instance. (Applies to individual relations and their attributes).
  - » Attributes of different entities (EMPLOYEEs, DEPARTMENTs, PROJECTs) should not be mixed in the same relation
  - » Only foreign keys should be used to refer to other entities
  - » Entity and relationship attributes should be kept apart as much as possible.
- Bottom Line: Design a schema that can be explained easily relation by relation. The semantics of attributes should be easy to interpret.

Figure 14.1 A simplified COMPANY relational database schema

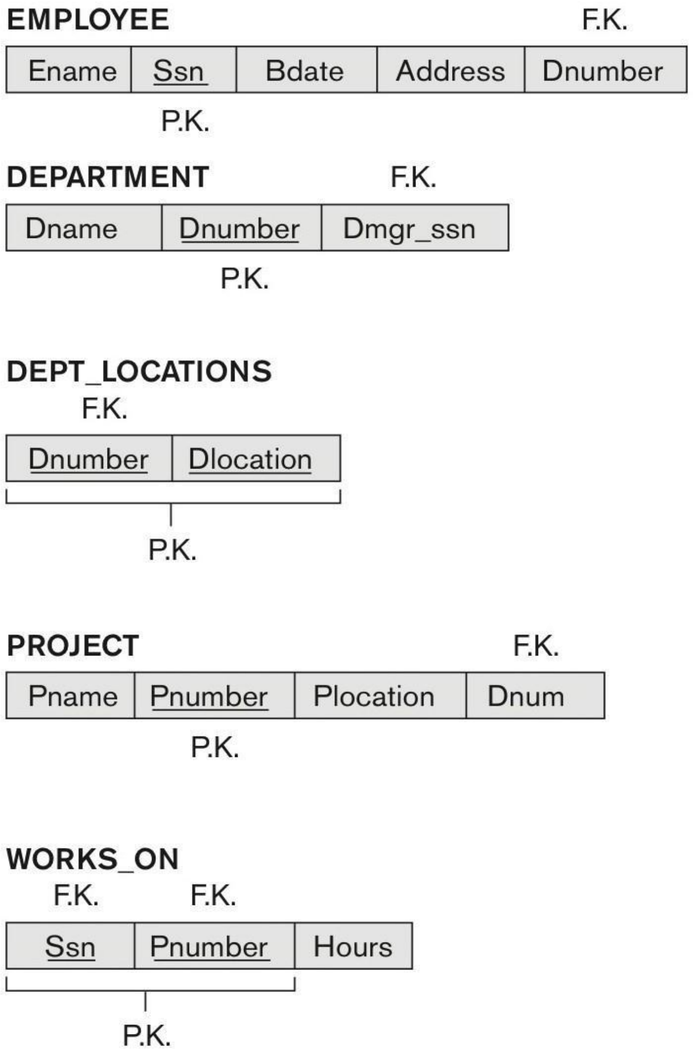


Figure 14.1 A simplified COMPANY relational database schema.

EMPLOYEE

Ename	Ssn	Bdate	Address	Dnumber
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4
Wallace, Jennifer S.	987654321	1941-06-20	291Berry, Bellaire, TX	4
Narayan, Ramesh K.	666884444	1962-09-15	975 Fire Oak, Humble, TX	5
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1

DEPARTMENT

Dname	Dnumber	Dmgr_ssn
Research	5	333445555
Administration	4	987654321
Headquarters	1	888665555

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

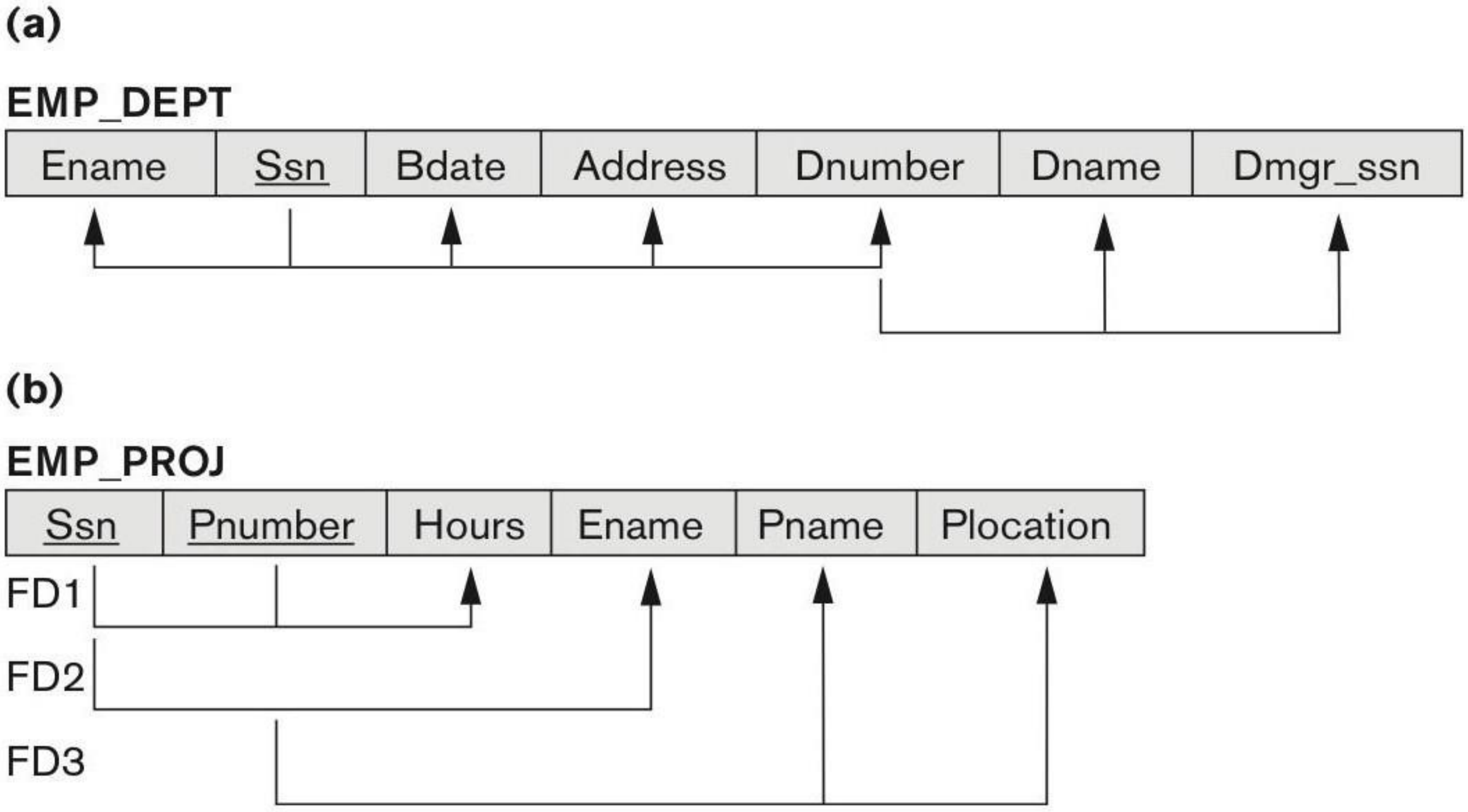
WORKS_ON

Ssn	Pnumber	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	Null

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

Examples of Violating Guideline 1



Sample states for EMP_DEPT and EMP_PROJ

Redundancy

EMP_DEPT

Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5	Research	333445555
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4	Administration	987654321
Wallace, Jennifer S.	987654321	1941-06-20	291 Berry, Bellaire, TX	4	Administration	987654321
Narayan, Ramesh K.	666884444	1962-09-15	975 FireOak, Humble, TX	5	Research	333445555
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5	Research	333445555
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4	Administration	987654321
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1	Headquarters	888665555

Redundancy Redundancy

EMP_PROJ

Ssn	Pnumber	Hours	Ename	Pname	Plocation
123456789	1	32.5	Smith, John B.	ProductX	Bellaire
123456789	2	7.5	Smith, John B.	ProductY	Sugarland
666884444	3	40.0	Narayan, Ramesh K.	ProductZ	Houston
453453453	1	20.0	English, Joyce A.	ProductX	Bellaire
453453453	2	20.0	English, Joyce A.	ProductY	Sugarland
333445555	2	10.0	Wong, Franklin T.	ProductY	Sugarland
333445555	3	10.0	Wong, Franklin T.	ProductZ	Houston
333445555	10	10.0	Wong, Franklin T.	Computerization	Stafford
333445555	20	10.0	Wong, Franklin T.	Reorganization	Houston
999887777	30	30.0	Zelaya, Alicia J.	Newbenefits	Stafford
999887777	10	10.0	Zelaya, Alicia J.	Computerization	Stafford
987987987	10	35.0	Jabbar, Ahmad V.	Computerization	Stafford
987987987	30	5.0	Jabbar, Ahmad V.	Newbenefits	Stafford
987654321	30	20.0	Wallace, Jennifer S.	Newbenefits	Stafford
987654321	20	15.0	Wallace, Jennifer S.	Reorganization	Houston
888665555	20	Null	Borg, James E.	Reorganization	Houston

Redundant Information in Tuples and Update Anomalies

- Information is stored redundantly
  - Wastes storage
  - Causes problems with update anomalies
    - Insertion anomalies
    - Deletion anomalies
    - Modification anomalies

Insertion Anomaly

- To insert a new employee tuple into EMP_DEPT, we must include either the attribute values for the department that the employee works for, or NULLs (if the employee does not work for a department as yet).
- For example, to insert a new tuple for an employee who works in department number 5, we must enter all the attribute values of department 5 correctly so that they are consistent with the corresponding values for department 5 in other tuples in EMP_DEPT.

Deletion Anomalies

- If we delete from EMP_DEPT an employee tuple that happens to represent the last employee working for a particular department, the information concerning that department is lost inadvertently from the database.

Modification Anomalies

- In EMP_DEPT, if we change the value of one of the attributes of a particular department—say, the manager of department 5—we must update the tuples of all employees who work in that department; otherwise, the database will become inconsistent.

GUIDELINE 2:

» Design a schema that does not suffer from the insertion, deletion and update anomalies.

» If there are any anomalies present, then note them so that applications can be made to take them into account.

Null Values in Tuples

- GUIDELINE 3:
  - » Relations should be designed such that their tuples will have as few NULL values as possible
  - » Attributes that are NULL frequently could be placed in separate relations (with the primary key)
- Reasons for nulls:
  - » Attribute not applicable or invalid
  - » Attribute value unknown (may exist)
  - » Value known to exist, but unavailable

Generation of Spurious Tuples

- Consider the two relation schemas EMP_LOCS and EMP_PROJ1 , which can be used instead of the single EMP_PROJ relation
- A tuple in EMP_LOCS means that the employee whose name is Ename works on at least one project located at Plocation.

EMP_LOCS

Ename	Plocation
Smith, John B.	Bellaire
Smith, John B.	Sugarland
Narayan, Ramesh K.	Houston
English, Joyce A.	Bellaire
English, Joyce A.	Sugarland
Wong, Franklin T.	Sugarland
Wong, Franklin T.	Houston
Wong, Franklin T.	Stafford
Zelaya, Alicia J.	Stafford
Jabbar, Ahmad V.	Stafford
Wallace, Jennifer S.	Stafford
Wallace, Jennifer S.	Houston
Borg, James E.	Houston

EMP_PROJ1

Ssn	Pnumber	Hours	Pname	Plocation
123456789	1	32.5	ProductX	Bellaire
123456789	2	7.5	ProductY	Sugarland
666884444	3	40.0	ProductZ	Houston
453453453	1	20.0	ProductX	Bellaire
453453453	2	20.0	ProductY	Sugarland
333445555	2	10.0	ProductY	Sugarland
333445555	3	10.0	ProductZ	Houston
333445555	10	10.0	Computerization	Stafford
333445555	20	10.0	Reorganization	Houston
999887777	30	30.0	Newbenefits	Stafford
999887777	10	10.0	Computerization	Stafford
987987987	10	35.0	Computerization	Stafford
987987987	30	5.0	Newbenefits	Stafford
987654321	30	20.0	Newbenefits	Stafford
987654321	20	15.0	Reorganization	Houston
888665555	20	NULL	Reorganization	Houston

	Ssn	Pnumber	Hours	Pname	Plocation	Ename
	123456789	1	32.5	ProductX	Bellaire	Smith, John B.
*	123456789	1	32.5	ProductX	Bellaire	English, Joyce A.
	123456789	2	7.5	ProductY	Sugarland	Smith, John B.
*	123456789	2	7.5	ProductY	Sugarland	English, Joyce A.
*	123456789	2	7.5	ProductY	Sugarland	Wong, Franklin T.
	666884444	3	40.0	ProductZ	Houston	Narayan, Ramesh K.
*	666884444	3	40.0	ProductZ	Houston	Wong, Franklin T.
*	453453453	1	20.0	ProductX	Bellaire	Smith, John B.
	453453453	1	20.0	ProductX	Bellaire	English, Joyce A.
*	453453453	2	20.0	ProductY	Sugarland	Smith, John B.
	453453453	2	20.0	ProductY	Sugarland	English, Joyce A.
*	453453453	2	20.0	ProductY	Sugarland	Wong, Franklin T.
*	333445555	2	10.0	ProductY	Sugarland	Smith, John B.
*	333445555	2	10.0	ProductY	Sugarland	English, Joyce A.
	333445555	2	10.0	ProductY	Sugarland	Wong, Franklin T.
*	333445555	3	10.0	ProductZ	Houston	Narayan, Ramesh K.
	333445555	3	10.0	ProductZ	Houston	Wong, Franklin T.
	333445555	10	10.0	Computerization	Stafford	Wong, Franklin T.
*	333445555	20	10.0	Reorganization	Houston	Narayan, Ramesh K.
	333445555	20	10.0	Reorganization	Houston	Wong, Franklin T.

Result of applying NATURAL JOIN to the tuples in EMP_PROJ1 and EMP_LOCS
just for employee with Ssn = “123456789”. Generated spurious tuples are marked by asterisks.

Generation of Spurious Tuples – avoid at any cost

- Bad designs for a relational database may result in erroneous results for certain JOIN operations
- The "lossless join" property is used to guarantee meaningful results for join operations
- GUIDELINE 4:
  - » The relations should be designed to satisfy the lossless join condition.
  - » No spurious tuples should be generated by doing a natural-join of any relations.

Summary and Discussion of Design Guidelines

- Anomalies that cause redundant work to be done during insertion into and modification of a relation, and that may cause accidental loss of information during a deletion from a relation
- Waste of storage space due to NULLs and the difficulty of performing selections, aggregation operations, and joins due to NULL values
- Generation of invalid and spurious data during joins on base relations with matched attributes that may not represent a proper (foreign key, primary key) relationship

2. Functional Dependencies

» Are used to specify formal measures of the "goodness" of relational designs

» And keys are used to define normal forms for relations

» Are constraints that are derived from the meaning and interrelationships of the data attributes

- A set of attributes X functionally determines a set of attributes Y if the value of X determines a unique value for Y

2.1 Defining Functional Dependencies

- $X \rightarrow Y$ holds if whenever two tuples have the same value for X, they *must have* the same value for Y
  - » For any two tuples t1 and t2 in any relation instance r(R): If t1[X]=t2[X], *then* t1[Y]=t2[Y]
- $X \rightarrow Y$ in R specifies a *constraint* on all relation instances r(R)
- Written as $X \rightarrow Y$; can be displayed graphically on a relation schema as in Figures. (denoted by the arrow:).
- FDs are derived from the real-world constraints on the attributes

Examples of FD constraints (1/2)

- Social security number determines employee name
  - » SSN → ENAME
- Project number determines project name and location
  - » PNUMBER → {PNAME, PLOCATION}
- Employee ssn and project number determines the hours per week that the employee works on the project
  - » {SSN, PNUMBER} → HOURS

Examples of FD constraints (2/2)

- An FD is a property of the attributes in the schema R
- The constraint must hold on *every* relation instance r(R)
- If K is a key of R, then K functionally determines all attributes in R
  - (since we never have two distinct tuples with t1[K]=t2[K])

Defining FDs from instances

- Note that in order to define the FDs, we need to understand the meaning of the attributes involved and the relationship between them.
- An FD is a property of the attributes in the schema R
- Given the instance (population) of a relation, all we can conclude is that an FD *may exist* between certain attributes.
- What we can definitely conclude is – that certain FDs *do not exist* because there are tuples that show a violation of those dependencies.

Ruling Out FDs

Note that given the state of the TEACH relation, we can say that the FD: Text → Course may exist. However, the FDs Teacher → Course, Teacher → Text and Course → Text are ruled out.

TEACH

Teacher	Course	Text
Smith	Data Structures	Bartram
Smith	Data Management	Martin
Hall	Compilers	Hoffman
Brown	Data Structures	Horowitz

What FDs may exist?

- A relation *R*(A, B, C, D) with its extension. Which FDs <u>*may exist*</u> in this relation?
- The following FDs may hold because the four tuples in the current extension have no violation of these constraints: B → C; C → B; {A, B} → C; {A, B} → D; and {C, D} → B.
- However, the following do not hold because we already have violations of them in the given extension: A → B (tuples 1 and 2 violate this constraint); B → A (tuples 2 and 3 violate this constraint); D → C (tuples 3 and 4 violate it).

A	B	C	D
a1	b1	c1	d1
a1	b2	c2	d2
a2	b2	c2	d3
a3	b3	c4	d3

3 Normal Forms Based on Primary Keys

- 3.1 Normalization of Relations
- 3.2 Practical Use of Normal Forms
- 3.3 Definitions of Keys and Attributes Participating in Keys
- 3.4 First Normal Form
- 3.5 Second Normal Form
- 3.6 Third Normal Form

3.1 Normalization of Relations (1/2)

» The process of decomposing unsatisfactory "bad" relations by breaking up their attributes into smaller relations

- **Normal form:**
  » Condition using keys and FDs of a relation to certify whether a relation schema is in a particular normal form

3.1 Normalization of Relations (2/2)

- 2NF, 3NF, BCNF
  - based on keys and FDs of a relation schema
- 4NF
  - based on keys, multi-valued dependencies : MVDs;
- 5NF
  - based on keys, join dependencies : JDs
- Additional properties may be needed to ensure a good relational design (lossless join, dependency preservation; see next section)

3.2 Practical Use of Normal Forms

- **Normalization** is carried out in practice so that the resulting designs are of high quality and meet the desirable properties

- The practical utility of these normal forms becomes questionable when the constraints on which they are based are *hard to understand* or to *detect*
- The database designers *need not* normalize to the highest possible normal form
  - (usually up to 3NF and BCNF. 4NF rarely used in practice.)
- **Denormalization**:
  - The process of storing the join of higher normal form relations as a base relation—which is in a lower normal form

3.3 Definitions of Keys and Attributes - Participating in Keys (1/2)

- A **superkey** of a relation schema R = {A1, A2, ..., An} is a set of attributes S *subset- of* R with the property that no two tuples t1 and t2 in any legal relation state r of R will have t1[S] = t2[S]

- A **key** K is a **superkey** with the *additional property* that removal of any attribute from K will cause K not to be a superkey any more.

3.3 Definitions of Keys and Attributes - Participating in Keys (2/2)

- If a relation schema has more than one key, each is called a **candidate** key.
  - » One of the candidate keys is *arbitrarily* designated to be the **primary key**, and the others are called **secondary keys**.
- A **Prime attribute** must be a member of *some* candidate key
- A **Nonprime attribute** is not a prime attribute—that is, it is not a member of any candidate key.

3.4 First Normal Form

- Disallows
  - composite attributes
  - multivalued attributes
  - **nested relations**; attributes whose values for an *individual tuple* are non-atomic
- Considered to be part of the definition of a relation
- Most RDBMSs allow only those relations to be defined that are in First Normal Form

Figure 14.9 Normalization into 1NF

(a)

DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocations

(b)

DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

(c)

DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocation
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

Figure 14.9
Normalization into 1NF. (a) A relation schema that is not in 1NF. (b) Sample state of relation DEPARTMENT. (c) 1NF version of the same relation with redundancy.

Figure 14.10 Normalizing nested relations into 1NF

(a)

EMP_PROJ

		Projs	
Ssn	Ename	Pnumber	Hours

(b)

EMP_PROJ

Ssn	Ename	Pnumber	Hours
123456789	Smith, John B.	1	32.5
		2	7.5
666884444	Narayan, Ramesh K.	3	40.0
453453453	English, Joyce A.	1	20.0
		2	20.0
333445555	Wong, Franklin T.	2	10.0
		3	10.0
		10	10.0
		20	10.0
999887777	Zelaya, Alicia J.	30	30.0
		10	10.0
987987987	Jabbar, Ahmad V.	10	35.0
		30	5.0
987654321	Wallace, Jennifer S.	30	20.0
		20	15.0
888665555	Borg, James E.	20	NULL

(c)

EMP_PROJ1

Ssn	Ename

EMP_PROJ2

Ssn	Pnumber	Hours

Figure 14.10 Normalizing nested relations into 1NF. (a) Schema of the EMP_PROJ relation with a nested relation attribute PROJS. (b) Sample extension of the EMP_PROJ relation showing nested relations within each tuple. (c) Decomposition of EMP_PROJ into relations EMP_PROJ1 and EMP_PROJ2 by propagating the primary key.

3.5 Second Normal Form (1/2)

- Uses the concepts of **FDs, primary key**
- Definitions
  - **Prime attribute:** An attribute that is member of the primary key K
  - **Full functional dependency:** a FD Y -> Z where removal of any attribute from Y means the FD does not hold any more
- Examples:
  - {SSN, PNUMBER} -> HOURS is a full FD since neither SSN -> HOURS nor PNUMBER -> HOURS hold
  - {SSN, PNUMBER} -> ENAME is not a full FD (it is called a partial dependency) since SSN -> ENAME also holds

3.5 Second Normal Form (2/2)

- A relation schema R is in **second normal form (2NF)** if every non-prime attribute A in R is fully functionally dependent on the primary key

- R can be decomposed into 2NF relations via the process of 2NF normalization or “second normalization”

Figure 14.11 Normalizing into 2NF and 3NF

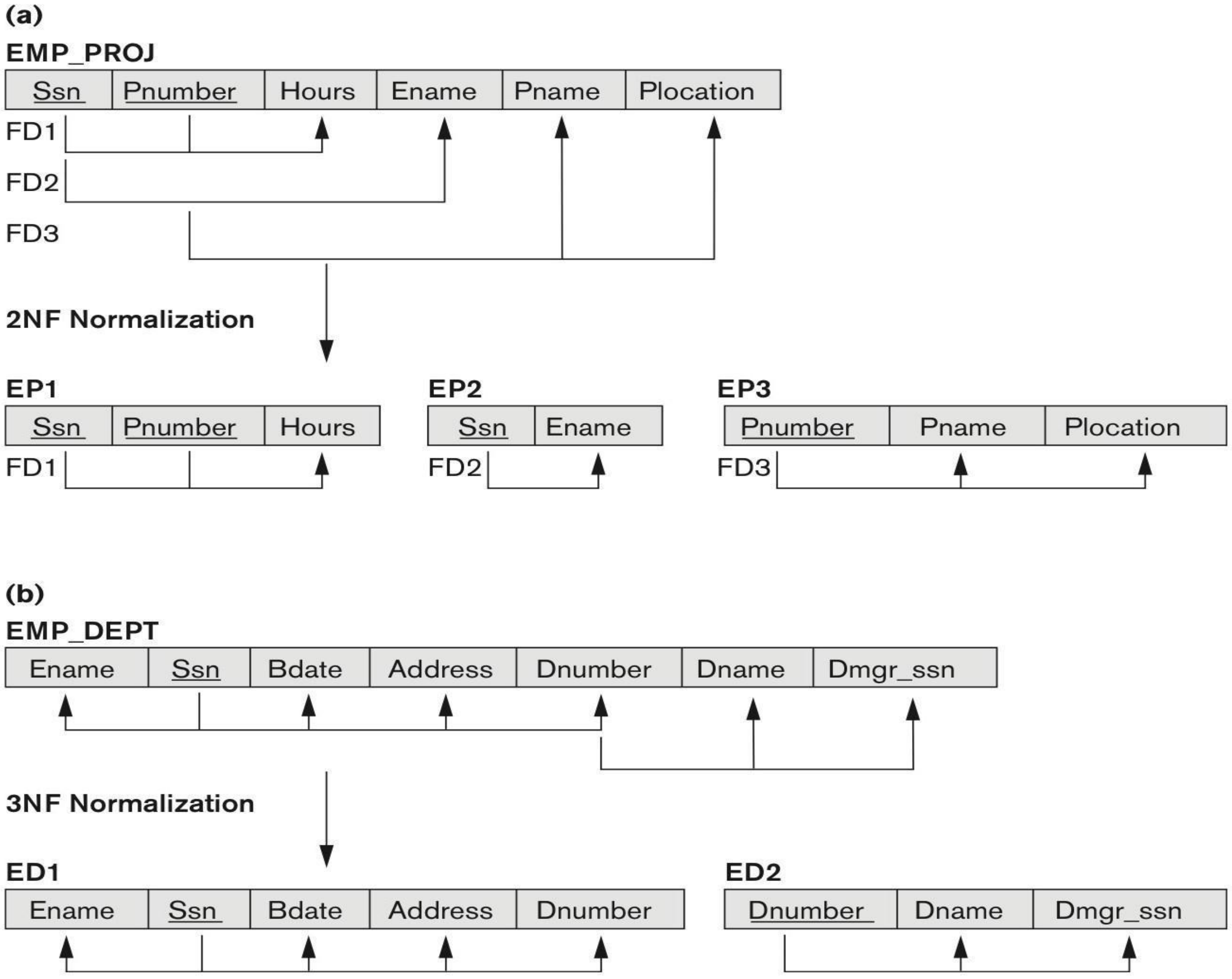


Figure 14.11
Normalizing into 2NF and 3NF.
(a) Normalizing EMP_PROJ into 2NF relations. (b) Normalizing EMP_DEPT into 3NF relations.

Third Normal Form (1/2)

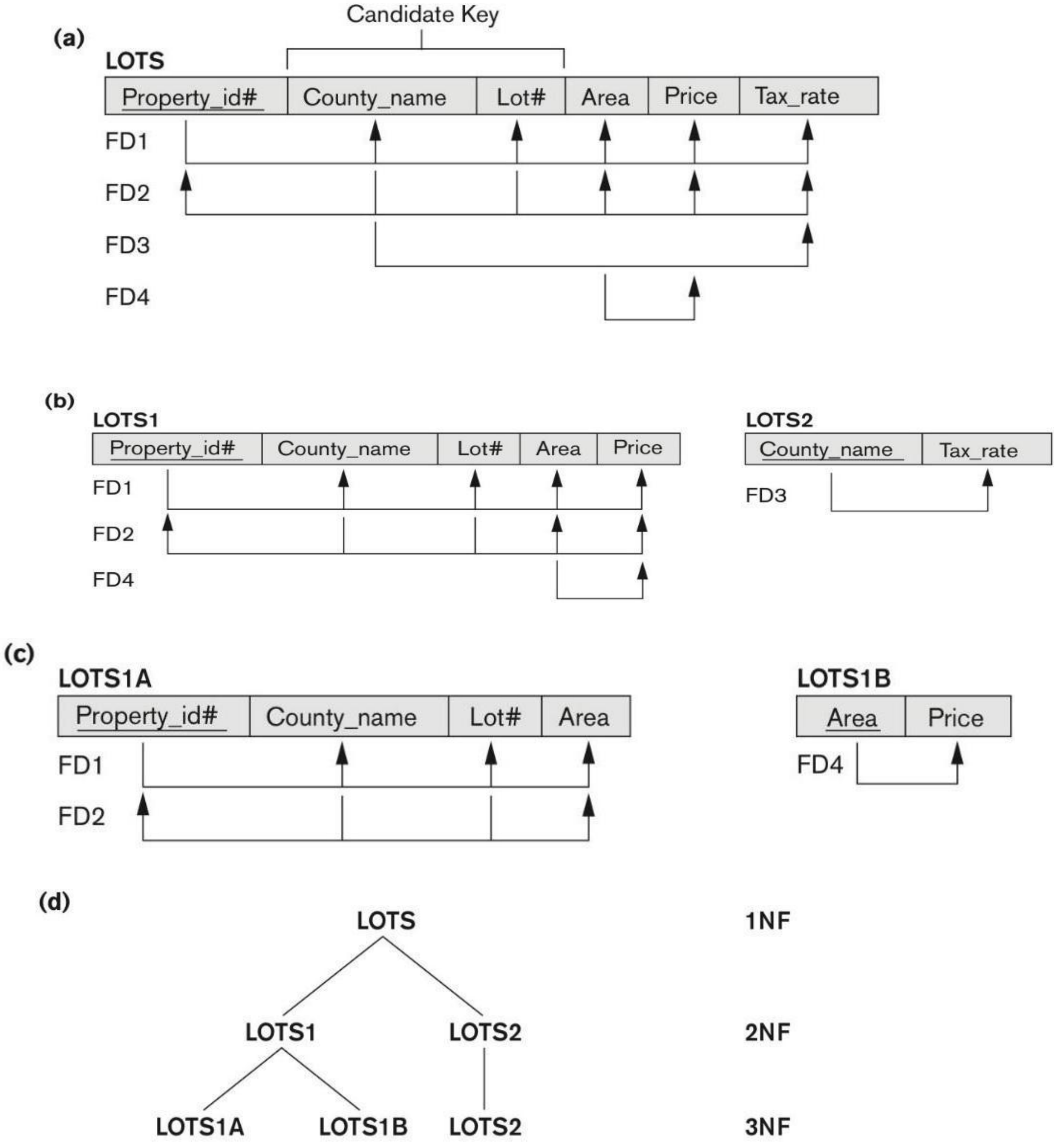
- Definition:
  - » **Transitive functional dependency:**

    a FD X - > Z that can be derived from two FDs

    X -> Y and Y -> Z
- Examples:
  - » SSN -> DMGRSSN is a **transitive** FD
    - Since SSN -> DNUMBER and DNUMBER -> DMGRSSN hold
  - » SSN -> ENAME is **non-transitive**
    - Since there is no set of attributes X where SSN -> X and X -> ENAME

Figure 14.12 Normalization into 2NF and 3NF

Figure 14.12 Normalization into 2NF and 3NF. (a) The LOTS relation with its functional dependencies FD1 through FD4. (b) Decomposing into the 2NF relations LOTS1 and LOTS2. (c) Decomposing LOTS1 into the 3NF relations LOTS1A and LOTS1B. (d) Progressive normalization of LOTS into a 3NF design.



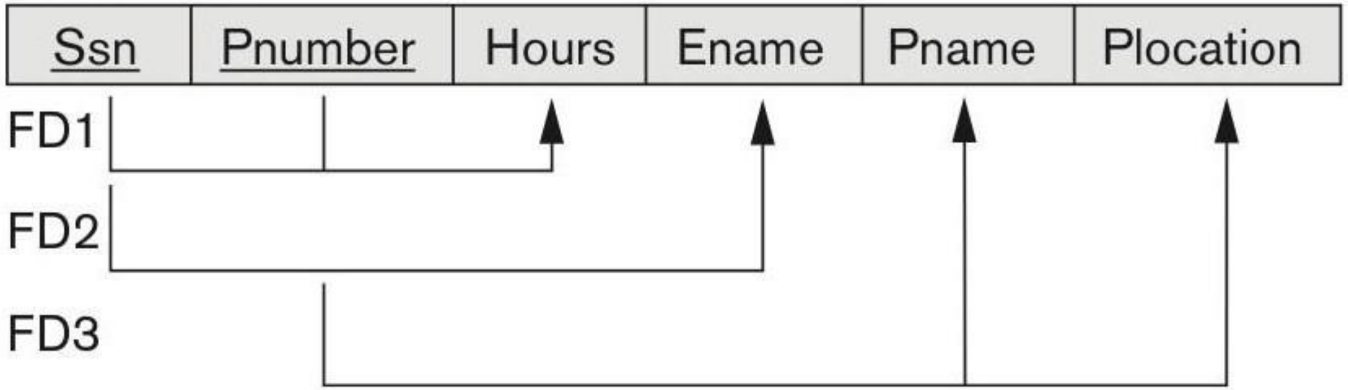
جامعة مصر للمعلوماتية
EGYPT UNIVERSITY
OF INFORMATICS

(a)

EMP_DEPT

(b)

EMP_PROJ



Third Normal Form (2/2)

- A relation schema R is in **third normal form (3NF)** if it is in 2NF *and* no non-prime attribute A in R is transitively dependent on the primary key
- R can be decomposed into 3NF relations via the process of 3NF normalization
- NOTE:
  - » In X -> Y and Y -> Z, with X as the primary key, we consider this a problem only if Y is not a candidate key.
  - » When Y is a candidate key, there is no problem with the transitive dependency .
  - » E.g., Consider EMP (SSN, Emp#, Salary).
    - Here, SSN -> Emp# -> Salary and Emp# is a candidate key.

Normal Forms Defined Informally

- 1st normal form
  - » All attributes depend on **the key**
- 2nd normal form
  - » All attributes depend on **the whole key**
- 3rd normal form
  - » All attributes depend on **nothing but the key**

4. General Normal Form Definitions (For Multiple Keys) (1/2)

- The above definitions consider the primary key only
- The following more general definitions take into account relations with multiple candidate keys
- Any attribute involved in a candidate key is a ***prime attribute***
- All other attributes are called ***non-prime attributes.***

4.1 General Definition of 2NF (For Multiple Candidate Keys) (2/2)

- A relation schema R is in **second normal form (2NF) if every non-prime attribute A in R is fully functionally dependent on *every* key of R**
- In Figure 14.12:the relation has **two candidate keys** Property_id# and {County_name, Lot#},
- So, the FD3 County_name → Tax_rate violates 2NF.
- As Tax_rate depends on part of the candidate key

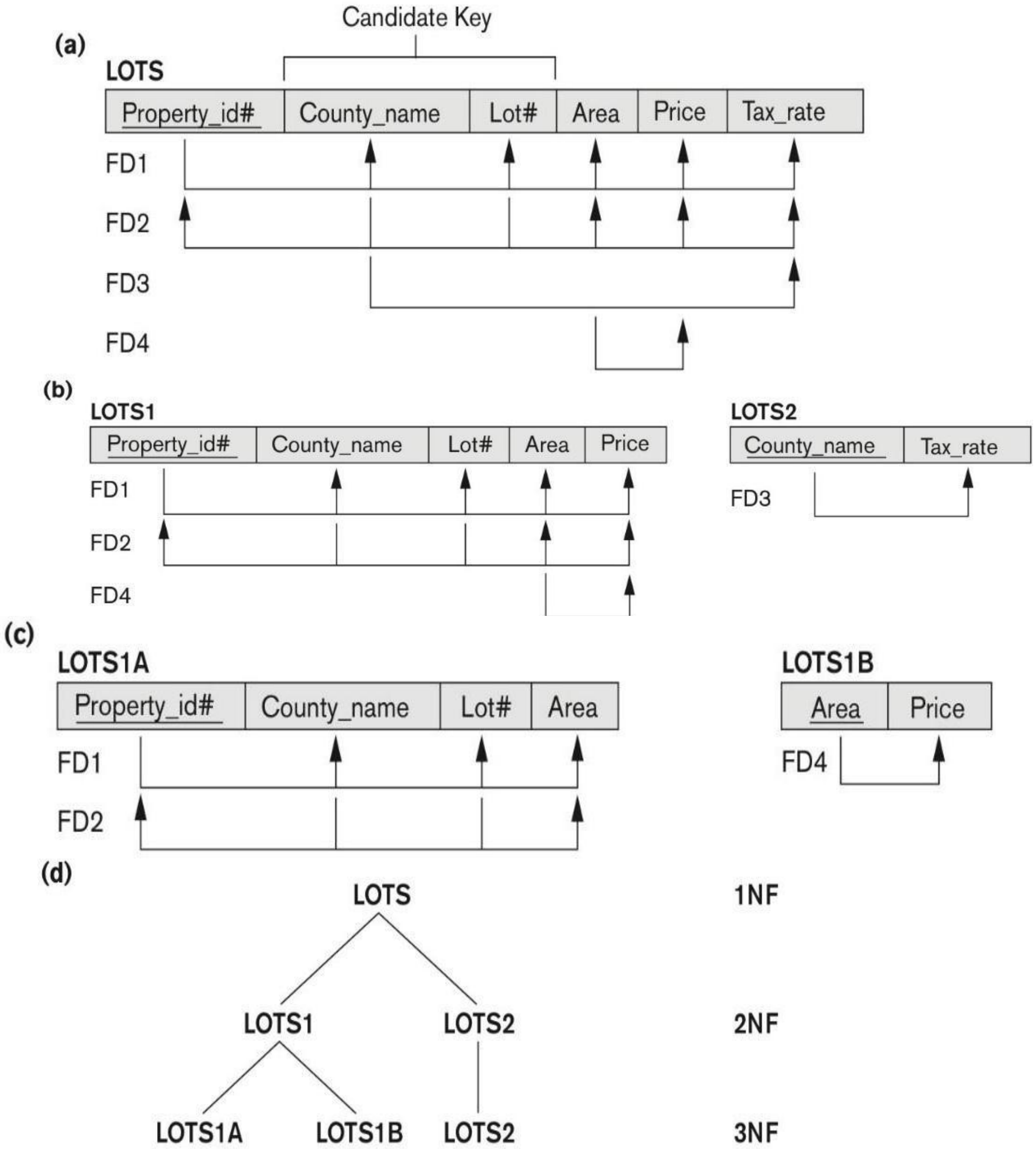
So second normalization converts LOTS into:

LOTS1 (Property_id#, County_name, Lot#, Area, Price)

LOTS2 (County_name, Tax_rate)

Figure 14.12 Normalization into 2NF and 3NF

Figure 14.12 Normalization into 2NF and 3NF. (a) The LOTS relation with its functional dependencies FD1 through FD4. (b) Decomposing into the 2NF relations LOTS1 and LOTS2. (c) Decomposing LOTS1 into the 3NF relations LOTS1A and LOTS1B. (d) Progressive normalization of LOTS into a 3NF design.



4.2 General Definition of Third Normal Form

- Definition:
  - » **Superkey** of relation schema R - a set of attributes S of R that contains a key of R
  - » A relation schema R is in **third normal form (3NF)** if whenever a FD X → A holds in R, then either:
    - (a) X is a superkey of R, or
    - (b) A is a prime attribute of R
- LOTS1 relation violates 3NF because

Area → Price ;   and Area is not a superkey in LOTS1.
(see Figure 14.12).

4.3 Interpreting the General Definition of Third Normal Form (1/2)

- Consider the 2 conditions in the Definition of 3NF:

  A relation schema R is in **third normal form (3NF)** if whenever a FD X → A holds in R, then either:

  - (a) X is a superkey of R, or
  - (b) A is a prime attribute of R

- Condition (a) catches two types of violations :

  - one where a prime attribute functionally determines a non- prime attribute. This catches 2NF violations due to non-full functional dependencies.

  -second, where a non-prime attribute functionally determines a non-prime attribute. This catches 3NF violations due to a transitive dependency.

4.3 Interpreting the General Definition of Third Normal Form (2/2)

- **ALTERNATIVE DEFINITION of 3NF: We can restate the definition as:**

  A relation schema R is in **third normal form (3NF)** if every non-prime attribute in R meets both of these conditions:

  » It is fully functionally dependent on every key of R

  » It is non-transitively dependent on every key of R Note that stated this way, a relation in 3NF also meets the requirements for 2NF.

- The condition (b) from the last slide takes care of the dependencies that “slip through” (are allowable to) 3NF but are “caught by” BCNF which we discuss next.

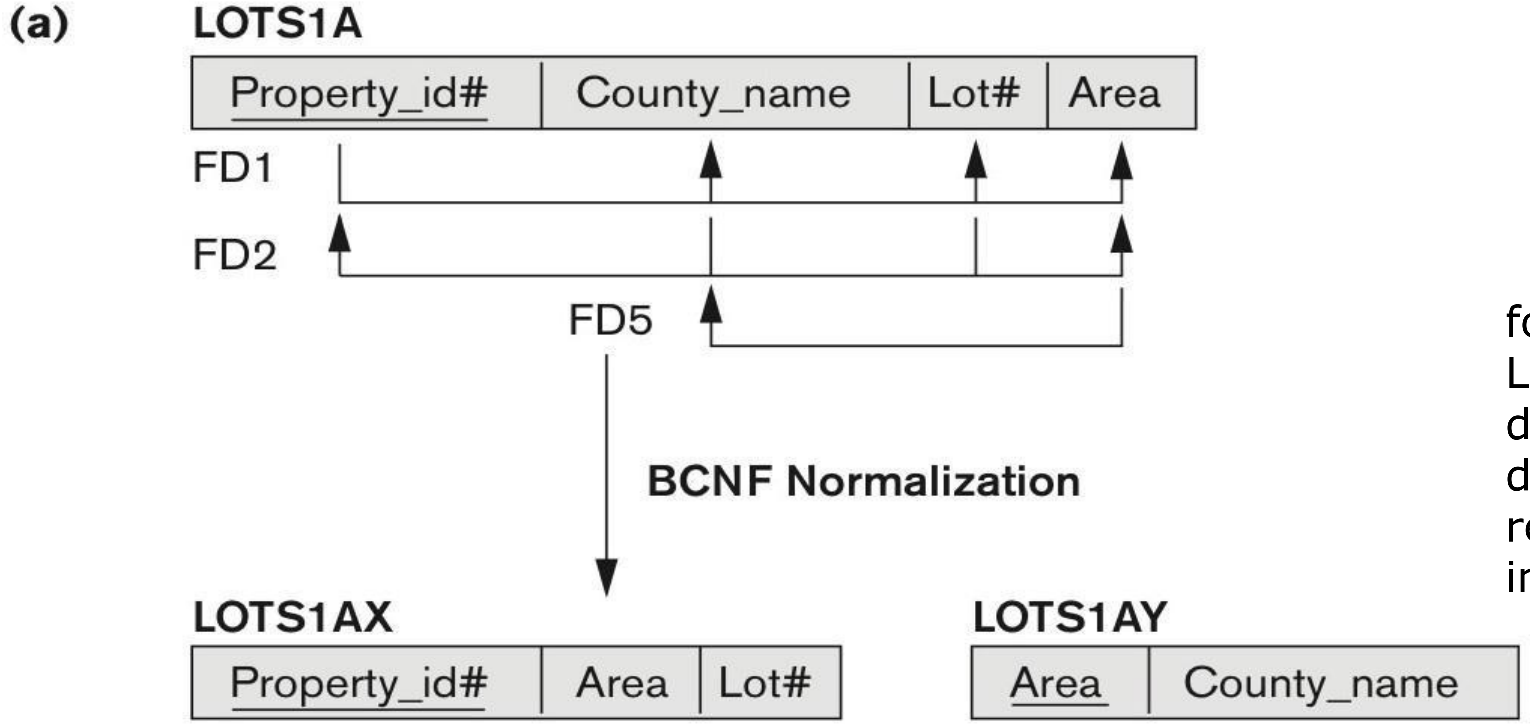
5. BCNF (Boyce-Codd Normal Form)

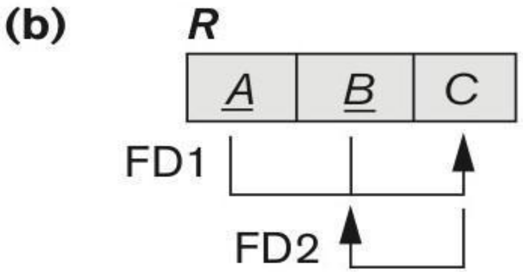
- A relation schema R is in **Boyce-Codd Normal Form (BCNF)** if whenever an **FD X ⟶ A** holds in R, then **X is a superkey** of R
- The formal definition of BCNF differs from the definition of 3NF in that clause (b) of 3NF, which allows f.d.'s having the RHS as a prime attribute, is absent from BCNF. That makes BCNF a stronger normal form compared to 3NF.

5. BCNF (Boyce-Codd Normal Form)

- Each normal form is strictly stronger than the previous one
  - » Every 2NF relation is in 1NF
  - » Every 3NF relation is in 2NF
  - » Every BCNF relation is in 3NF
- There exist relations that are in 3NF but not in BCNF
- Hence BCNF is considered a stronger form of 3NF
- The goal is to have each relation in BCNF (or 3NF)

Figure 14.13 Boyce-Codd normal form





FD1: A,B → C, FD2: C → B
R1 = (R - B), R2 = (C,B)
(C) BCNF Normalization of (b)

Figure 14.13 Boyce-Codd normal form. (a) BCNF normalization of LOTS1A with the functional dependency FD2 being lost in the decomposition. (b) A schematic relation with FDs; it is in 3NF, but not in BCNF due to the f.d. C → B.

Figure 14.14 A relation TEACH that is in 3NF but not in BCNF

TEACH

Student	Course	Instructor
Narayan	Database	Mark
Smith	Database	Navathe
Smith	Operating Systems	Ammar
Smith	Theory	Schulman
Wallace	Database	Mark
Wallace	Operating Systems	Ahamad
Wong	Database	Omiecinski
Zelaya	Database	Navathe
Narayan	Operating Systems	Ammar

Figure 14.14
A relation TEACH that is in 3NF but not BCNF.

- fd1: { student, course} -> instructor
- fd2: instructor -> course

BCNF (slide 61):

- *R1* = *(R* - course*)* = *(*student, instructor*)*
- *R2* = *(*instructor, course*)*

Achieving the BCNF by Decomposition (1/2)

- Two FDs exist in the relation TEACH:
  - » fd1: { student, course} -> instructor
  - » fd2: instructor -> course
- {student, course} is a candidate key for this relation and that the dependencies shown follow the pattern in Figure 14.13 (b).
  - » So this relation is in 3NF *but not in* BCNF
- A relation **NOT** in BCNF should be decomposed so as to meet this property, while possibly forgoing the preservation of all functional dependencies in the decomposed relations.
  - » (See Algorithm 15.3)

Achieving the BCNF by Decomposition (2/2)

- Three possible decompositions for relation TEACH
  - D1: {<u>student, instructor</u>} and {<u>student, course</u>}
  - D2: {course, <u>instructor</u> } and {<u>course, student</u>}
  - D3: {<u>instructor</u>, course } and {<u>instructor, student</u>} ✓
- All three decompositions will lose fd1.
  - We have to settle for sacrificing the functional dependency preservation. But we <u>cannot</u> sacrifice the non-additivity property after decomposition.
- Out of the above three, only the 3rd decomposition will not generate spurious tuples after join.(and hence has the non-additivity property).

- A test to determine whether a binary decomposition (decomposition into two relations) is non-additive (lossless) is discussed under Property NJB on the next slide. We then show how the third decomposition above meets the property.

Test for checking non-additivity of Binary Relational Decompositions

- **Testing Binary Decompositions for Lossless Join (Non-additive Join) Property**
  - **Binary Decomposition:** Decomposition of a relation R into two relations.
  - **PROPERTY NJB (non-additive join test for binary decompositions):** A decomposition D = {R1, R2} of R has the lossless join property with respect to a set of functional dependencies F on R *if and only if* either
    - The f.d. ((R1 ∩ R2) → (R1- R2)) is in F^+, or
    - The f.d. ((R1 ∩ R2) → (R2 - R1)) is in F^+.
- The notation F+ refers to the cover of the set of functional dependencies and includes all f.d.'s implied by F.

Test for checking non-additivity of Binary Relational Decompositions

- Two FDs exist in the relation TEACH:
  - » fd1: { student, course} -> instructor
  - » fd2: instructor -> course
- If you apply the NJB test to the 3 decompositions of the TEACH relation:
- D1 gives **Student** → Instructor or **Student** → Course
  - R1 (**Student**, Instructor) and R2(**Student**, Course)
  - R1 ∩ R2 = Student, (R1- R2) = Instructor, (R2- R1) = Course
  - Neither Student → Instructor, nor Student → Course is true.

Test for checking non-additivity of Binary Relational Decompositions

- D2 gives **Course** → Instructor or **Course** → Student
  - R1 (Course, Instructor) and R2(Course, Student)
  - R1 ∩ R2 = Course, (R1- R2) = Instructor, (R2- R1) = Student
  - Neither Student → Instructor, nor Student → Course is true.
- However, in D3 we get **Instructor** →Course or **Instructor** → Student
  - R1 (**Instructor**, Course) and R2(**Instructor**, Student)
  - R1 ∩ R2 = **Instructor**, (R1- R2) = Course, (R2- R1) = Student

Since **Instructor** → Course is indeed true, the NJB property is satisfied and D3 is determined as a non-additive (good) decomposition.

General Procedure for achieving BCNF when a relation fails BCNF

- Here we make use the following algorithm):
- Let R be the relation not in BCNF, let X be a subset-of R, and let $X \rightarrow A$ be the FD that causes a violation of BCNF.
- Then R may be decomposed into two relations:
  - (i) *R –A* and (ii) *X* **U** *A*.
  - If either *R –A* or *X* ∪ *A*. is not in BCNF, repeat the process.
- In Teach relation: **R (student, course, Instructor)**, the f.d. that violated BCNF $X \rightarrow A$ ***is* Instructor → Course**, **X(Instructor), A (Course)**
  - **R1 = R –A = (student, Instructor)**
  - **R2 = X U A = (Instructor, Course)**

    that we obtained before in decomposition D3.

Summary

- Informal Design Guidelines for Relational Databases
- Functional Dependencies (FDs)
- Normal Forms (1NF, 2NF, 3NF)Based on Primary Keys
- General Normal Form Definitions of 2NF and 3NF (For Multiple Keys)
- BCNF (Boyce-Codd Normal Form)

References

- Fundamentals of Database Systems (7th Edition) Ramez Elmasri and Shamkant Navathe , Ch. 14.

Questions